

Data Analysis

Exploratory Data Analysis and Visualization

Department of Economics

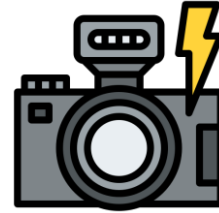
Ivan Bondarev



Email

- i.bondarev@inno.mgimo.ru

Signs



– Please take pictures of the information. Medium priority.

Source: https://www.flaticon.com/ru/free-icon/camera-flash_3460307



– Please write down the information. High priority.

Source: <https://openmoji.org/library/emoji-270D/>

TOPICS

1. Why draw the data
2. One variable: histogram, density, box plot, bar chart
3. Two variables: scatter, box, violin, stacked bar, heatmap
4. Several variables: correlation heatmap, pair plot
5. Several charts in one figure

Useful sources

- Github - Nikita Artamonov

<https://github.com/artamonoff>



Useful sources

- Github - Ivan Bondarev

<https://github.com/BondarevIM/MGIMO-SGIA-Data-Analysis>



From numbers to pictures

*Seminar 2: price skewness 0.66, no outliers,
correlation of carat and price 0.94.*

Can you picture what these numbers look like?

Why draw the data

- A few numbers summarise a column, a chart shows all of it
- The eye finds tails, gaps, peaks and groups at once
- Exploratory data analysis (EDA): look first, then ask questions
- Every chart in this seminar has a number from seminar 2 behind it
- A chart shows what is in the data, not why it is there

The dataset: Diamond.csv

- The same file as in seminar 2: 308 stones, 5 columns
 - o numerical: carat (weight), price (Singapore dollars)
 - o categorical: colour, clarity, certification
- Two of the categories are ordered scales:
 - o colour: D, E, F, G, H, I (D is the best)
 - o clarity: IF, VVS1, VVS2, VS1, VS2 (IF is the best)

Source: <https://jse.amstat.org/v9n2/datasets.chu.html>

Three libraries

```
import pandas as pd  
import matplotlib.pyplot as plt  
import seaborn as sns
```

- pandas holds the table
- matplotlib draws the figure: size, title, axis labels
- seaborn draws statistical charts in one line

Choosing a chart

- One numerical: histogram, density plot, box plot
- One categorical: bar chart
- Numerical and numerical: scatter plot
- Categorical and numerical: box plots or violins by group
- Categorical and categorical: stacked bar chart, heatmap
- Three or more: correlation heatmap, pair plot

Fix the order first

```
colour_order = ["D", "E", "F", "G", "H", "I"]  
clarity_order = ["IF", "VVS1", "VVS2", "VS1", "VS2"]
```

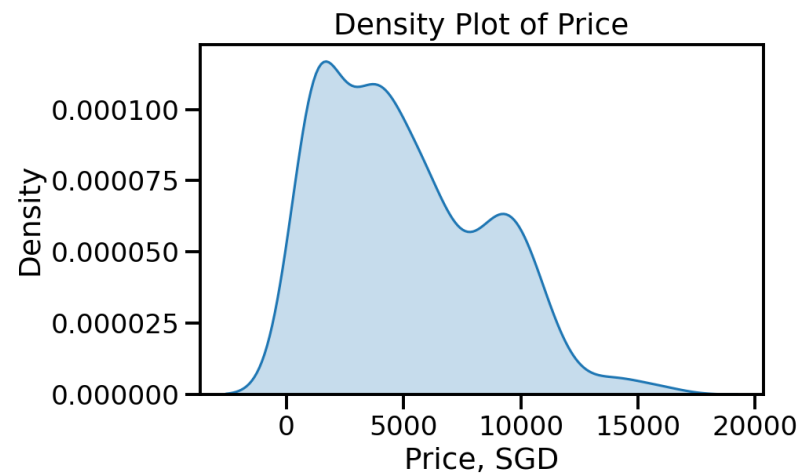
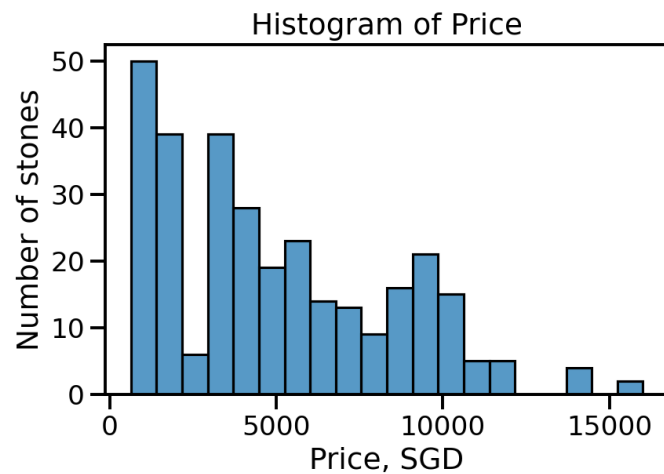
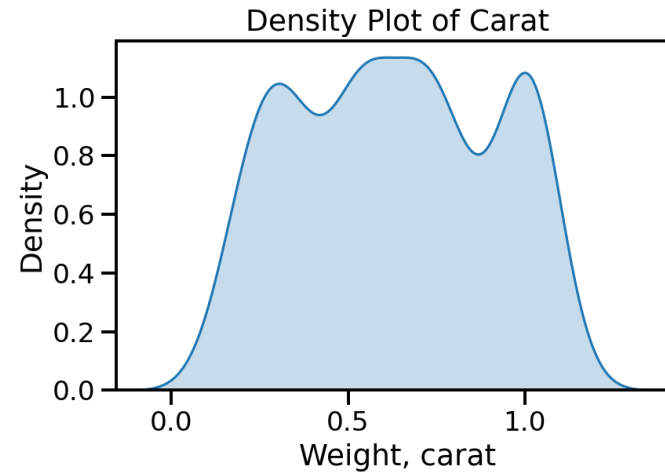
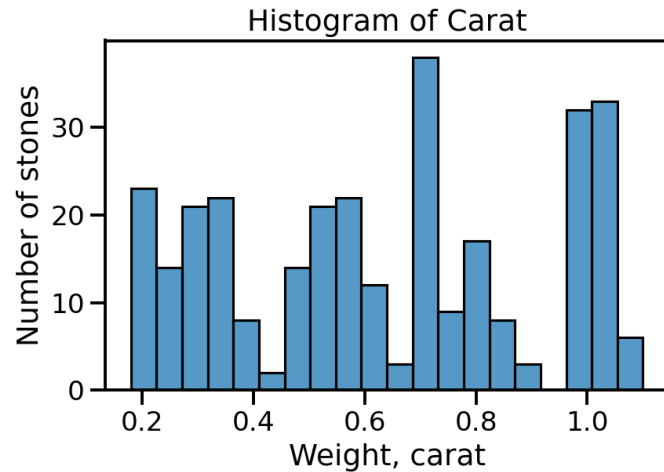
- pandas does not know that D is better than I
- Left alone, it sorts the categories by frequency: F, G, H, E, I, D
- Write the order once and reuse it in every chart
- certification has no natural order and needs no list

Histogram and density plot

```
sns.histplot(df["carat"], bins=20)  
sns.kdeplot(df["price"], fill=True)
```

- Histogram: the range is cut into 20 intervals (bins), each bar counts the stones in one
- Density plot: the same picture as a smooth outline
- fill=True shades the area under the outline

Reading a histogram



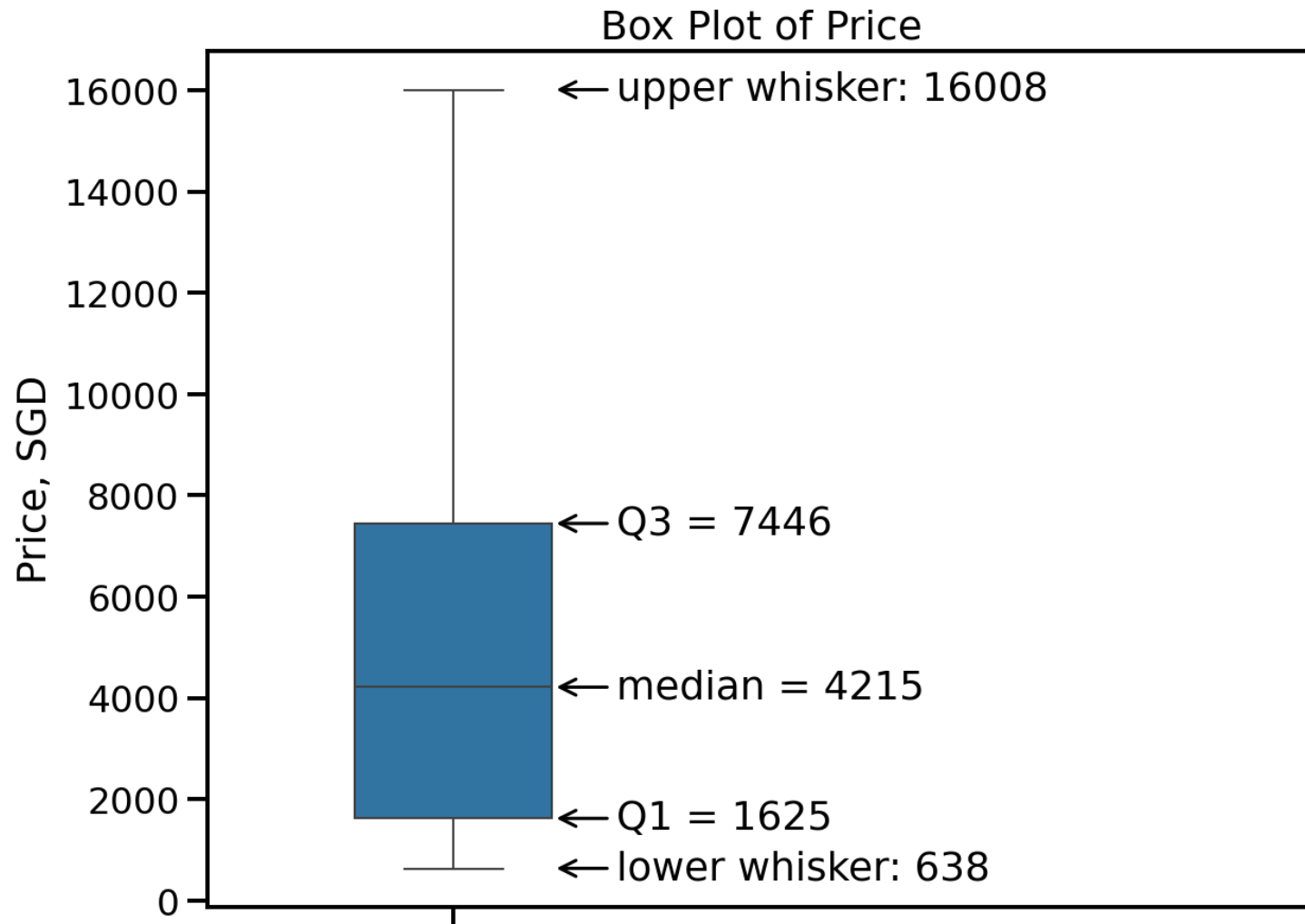
- Price has a long tail to the right: many cheap stones, a few very expensive ones
- This is the skewness of 0.66 from seminar 2
- Carat has peaks at 0.5, 0.7 and 1.0: the trade prefers round weights
- *The outline below zero is smoothing, not data*

Box plot

```
sns.boxplot(y=df["price"])
```

- The line inside the box: the median
- The box: from Q1 to Q3, the middle half of the stones (IQR)
- The whiskers: the most extreme stones within $1.5 \times \text{IQR}$ of the box
- A dot beyond a whisker: an outlier by the rule from seminar 2

Reading a box plot



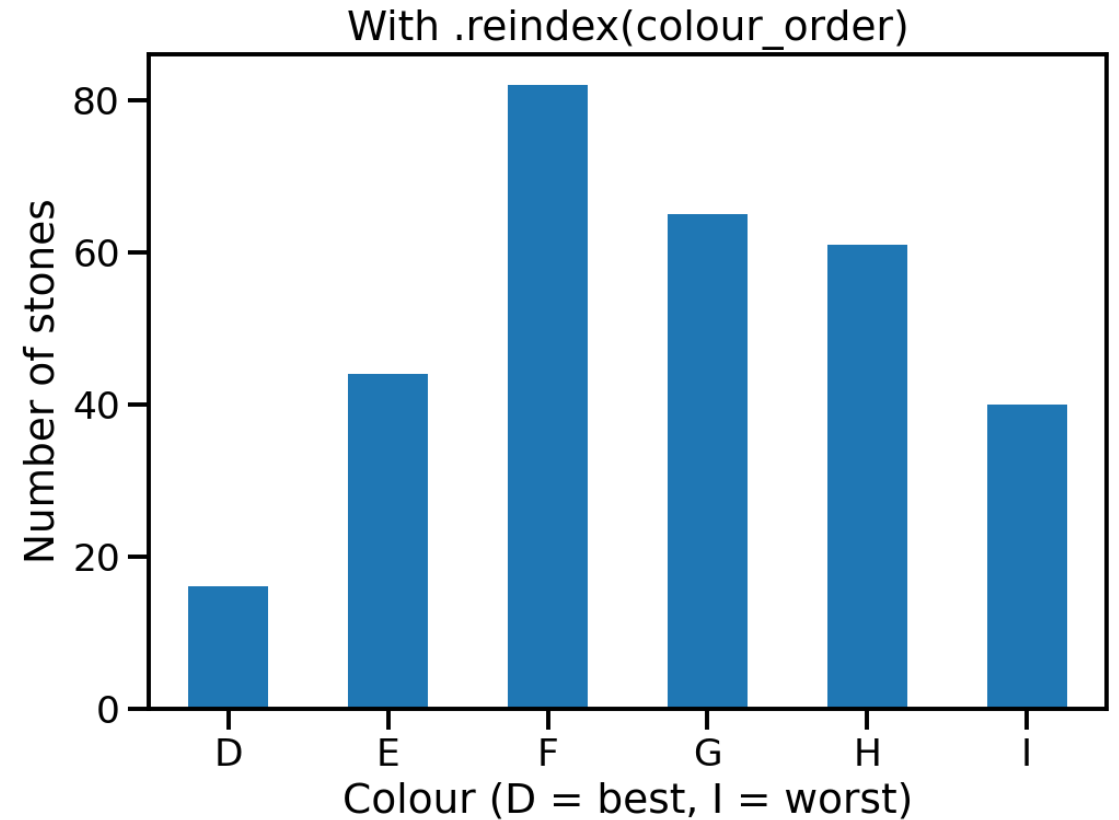
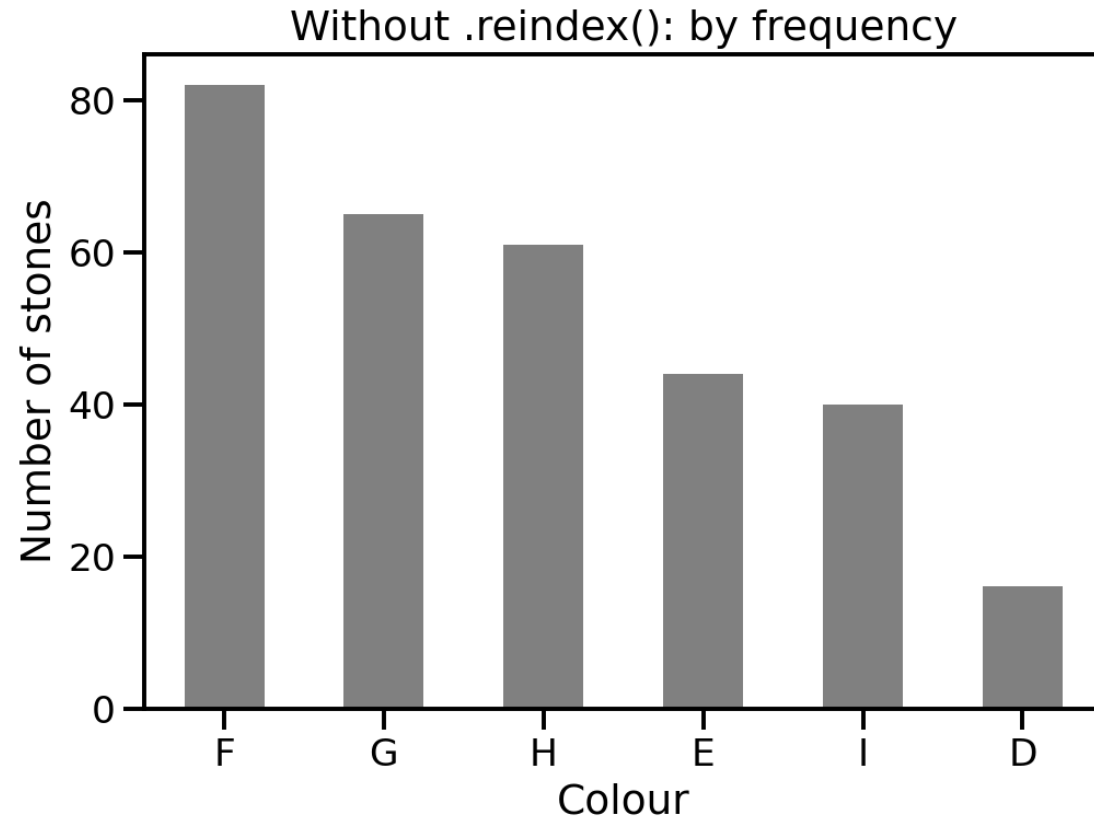
- No dots: no outliers, as in seminar 2
- Upper bound 16,177.50, most expensive stone 16,008
- So here the whiskers end at the minimum and the maximum
- Long upper whisker: the right tail again

Bar chart in the correct order

```
df["colour"].value_counts().reindex(colour_order).plot(kind="bar")  
plt.xticks(rotation=0)
```

- `.value_counts()` counts the stones in each category
- `.reindex(colour_order)` puts the bars in the order of the list
- `rotation=0` keeps the short labels horizontal

Why the order matters



Left: sorted by frequency, the quality scale is scrambled. Right: D to I, best to worst

Two variables together

IF is the best clarity a diamond can have.

Are IF diamonds the most expensive?

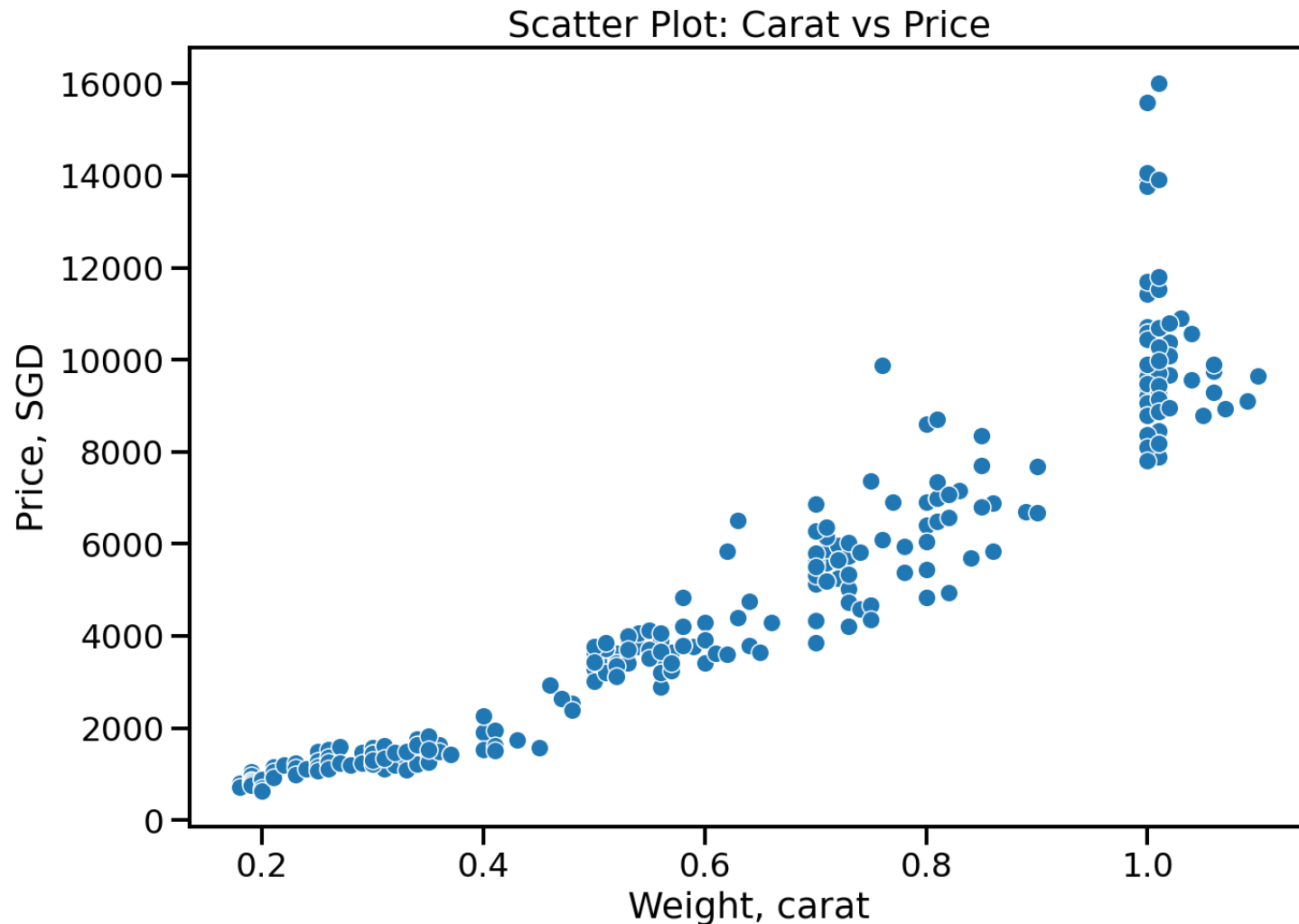
How would you check it with a chart?

Scatter plot

```
sns.scatterplot(data=df, x="carat", y="price")
```

- Each dot is one diamond
- Weight on the horizontal axis, price on the vertical axis
- Used for two numerical variables

Reading a scatter plot



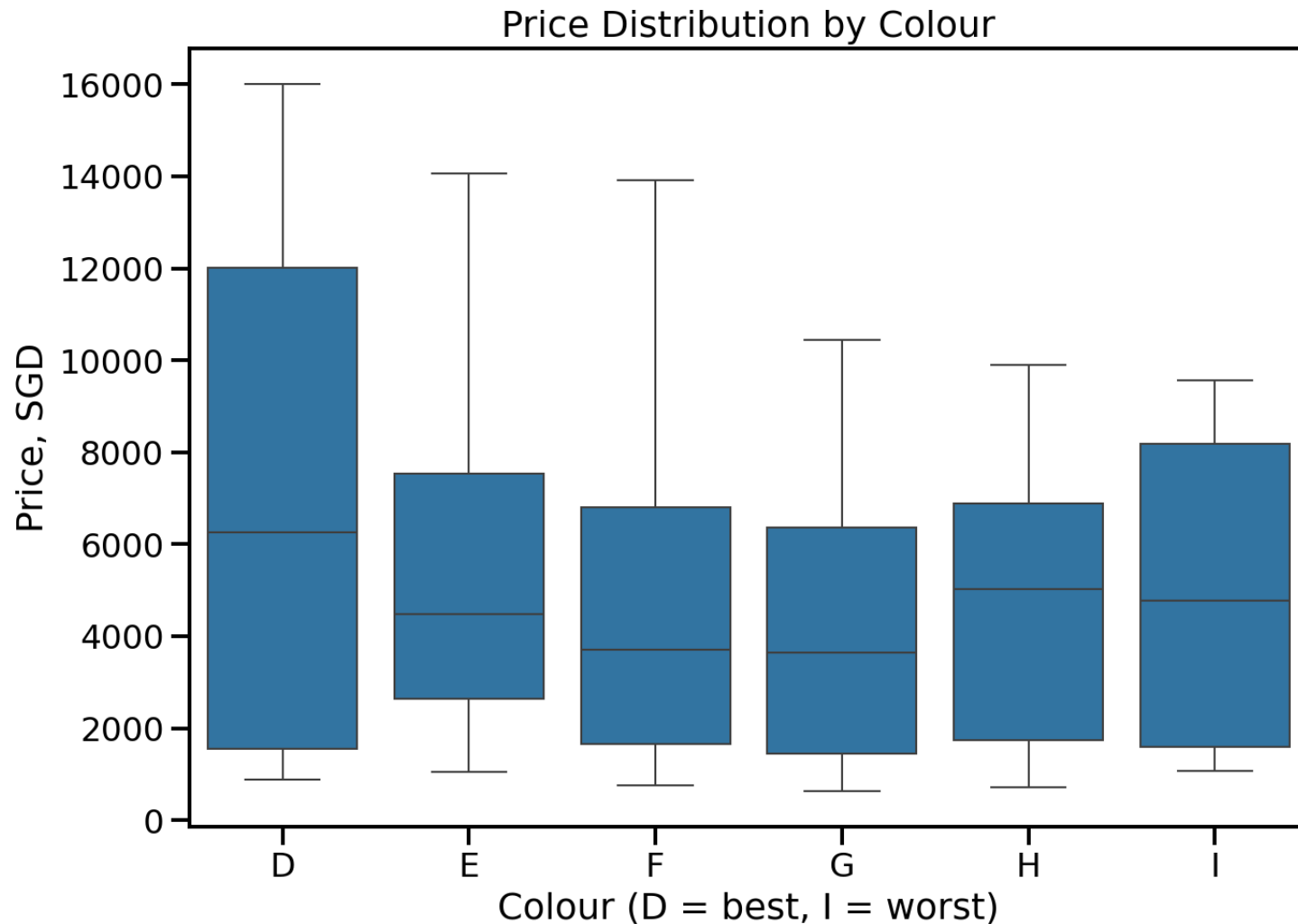
- The cloud rises: heavier stones cost more
- It is narrow: correlation 0.94 from seminar 2
- Vertical stripes at 0.5, 0.7 and 1.0 carat: the round weights
- It widens to the right: heavy stones vary more in price

Box plots by group

```
sns.boxplot(data=df, x="colour", y="price", order=colour_order)
```

- One box for each colour
- `order=colour_order` puts the boxes from D to I
- Without `order=` seaborn uses the order of the file

Price by colour



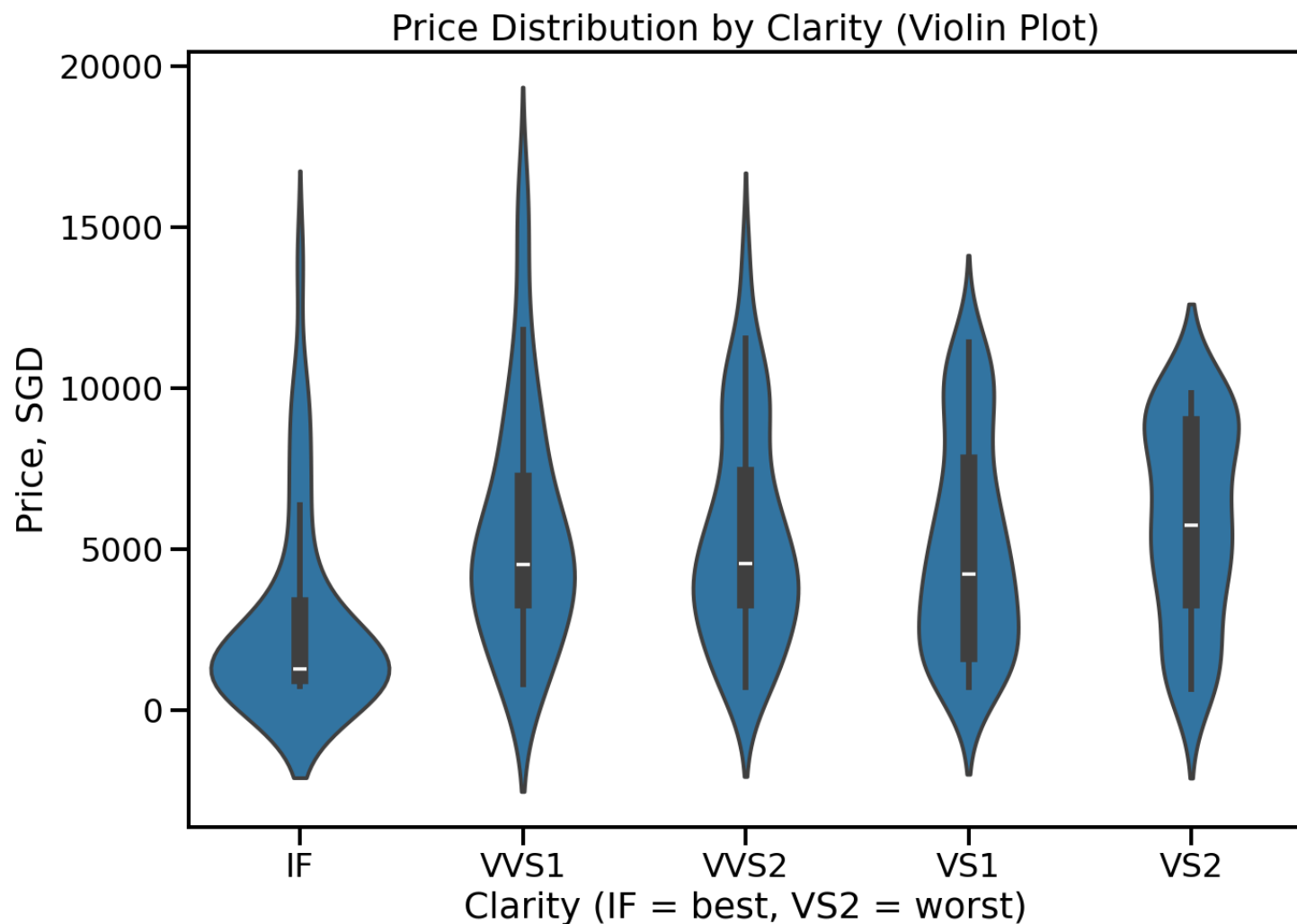
- Medians: D 6,266, E 4,484, F 3,714, G 3,651, H 5,030, I 4,780
- The worst colours are not the cheapest
- Weight explains it: median H and I stones weigh 0.71 and 0.78 carat, F and G only 0.55 and 0.57
- *Visible only because the boxes are in order*

Violin plot

```
sns.violinplot(data=df, x="clarity", y="price", order=clarity_order)
```

- A box plot with the density outline on both sides
- The widest part: where most stones of the group are priced
- `order=` works exactly as in the box plot

Price by clarity



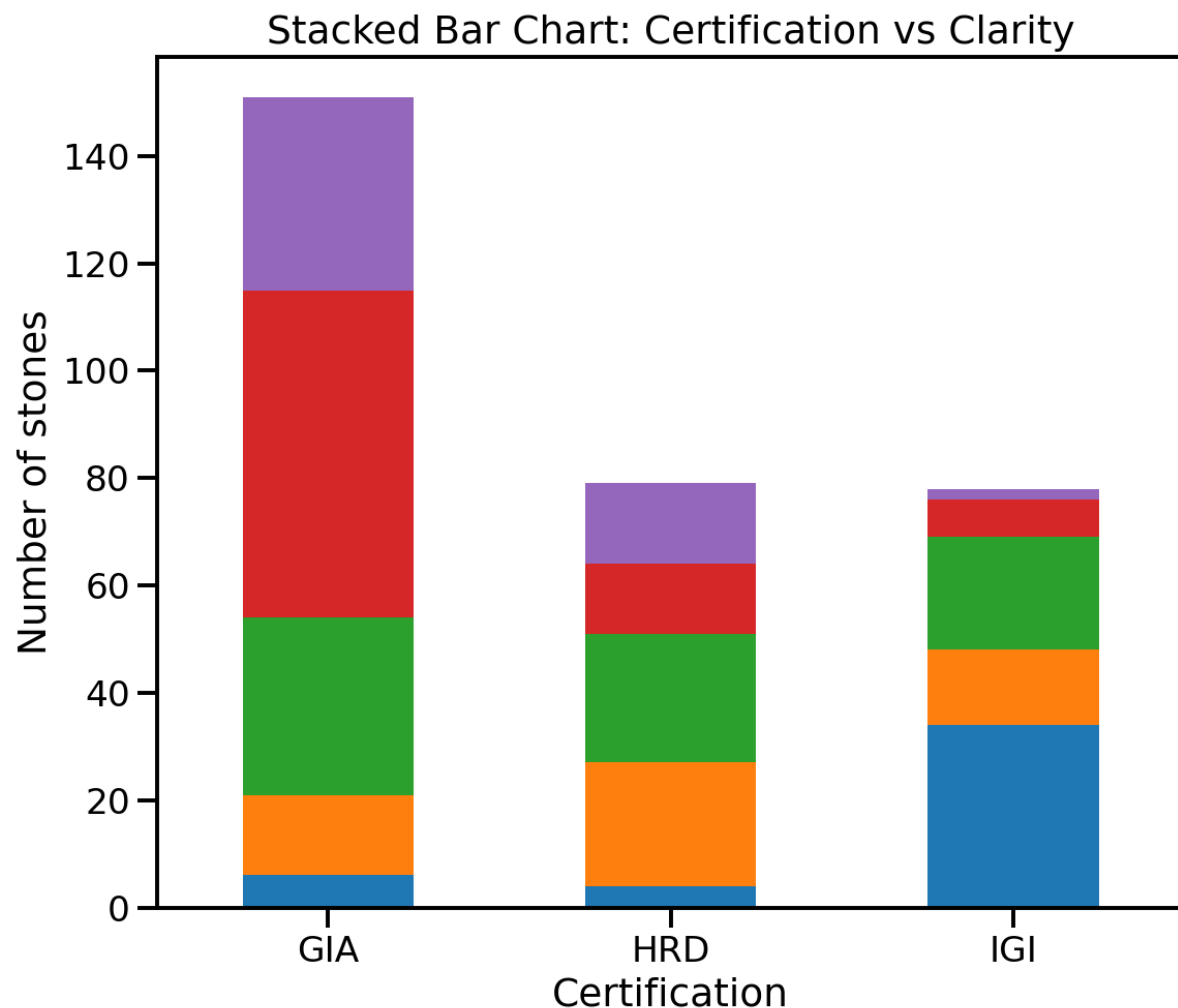
- IF, the best clarity, is the cheapest group: median 1,266 SGD
- VS2, the worst, is the most expensive: median 5,738 SGD
- Weight again: median IF stone 0.27 carat, median VS2 stone 0.82
- *A chart shows what is in the data, not the effect of one feature alone*

Stacked bar chart

```
cert_clarity = pd.crosstab(df["certification"],  
                           df["clarity"])[clarity_order]  
cert_clarity.plot(kind="bar", stacked=True)  
plt.legend(title="Clarity",  
           bbox_to_anchor=(1.05, 1), loc="upper left")
```

- `pd.crosstab` counts the stones in every pair of categories
- `[clarity_order]` puts the columns in our order instead of A to Z
- `bbox_to_anchor` moves the legend outside the chart

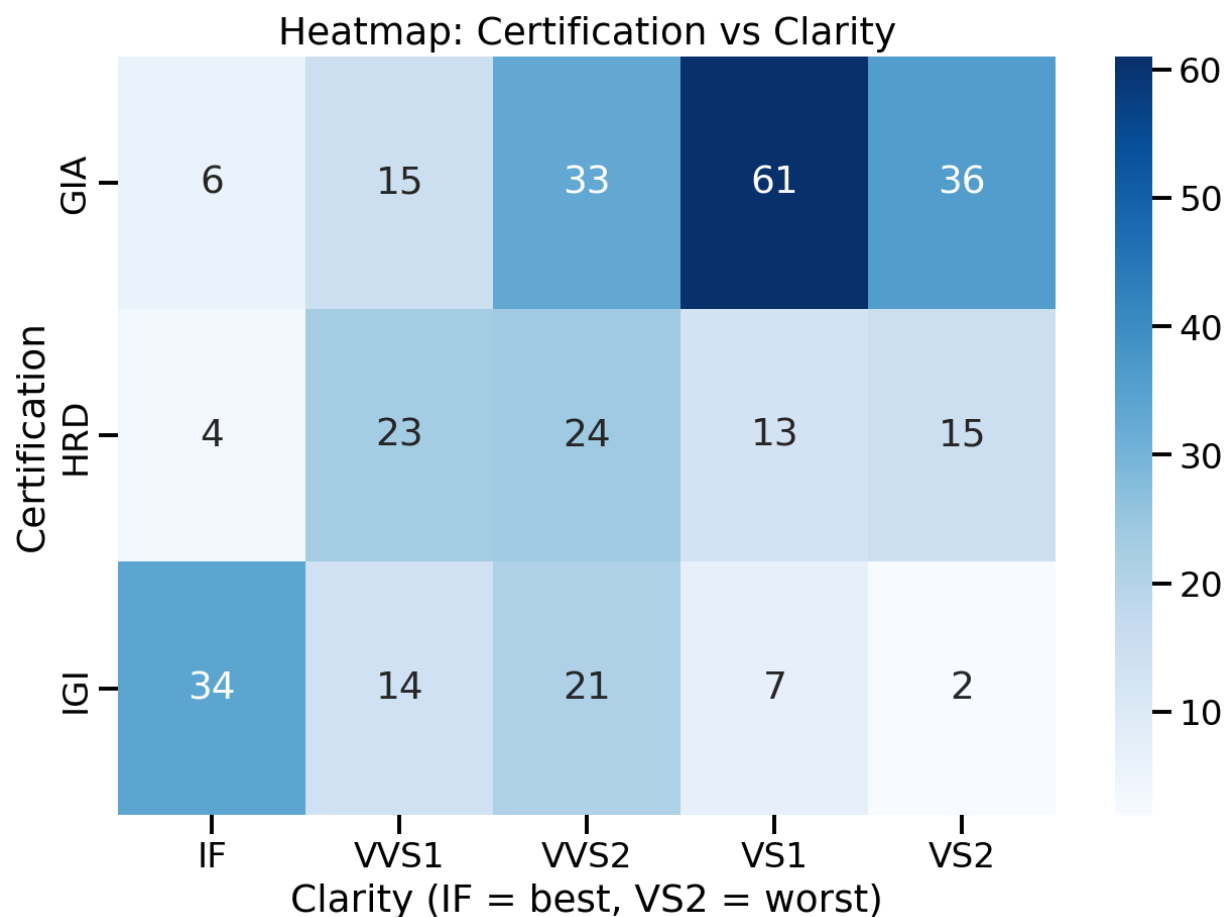
Certification against clarity



- Each bar is one laboratory, its pieces are clarity grades
- IGI: 34 of its 78 stones are IF
- GIA: the largest group is VS1, 61 of 151
- The laboratories certify very different stones

The same table as a heatmap

```
sns.heatmap(cert_clarity, annot=True, fmt="d", cmap="Blues")
```



- `annot=True` writes the counts in the cells
- `fmt="d"` prints them as whole numbers
- The darker the cell, the more stones
- Darkest: GIA-VS1 (61) and IGI-IF (34)

A third numerical variable

```
df["price_per_carat"] = df["price"] / df["carat"]
```

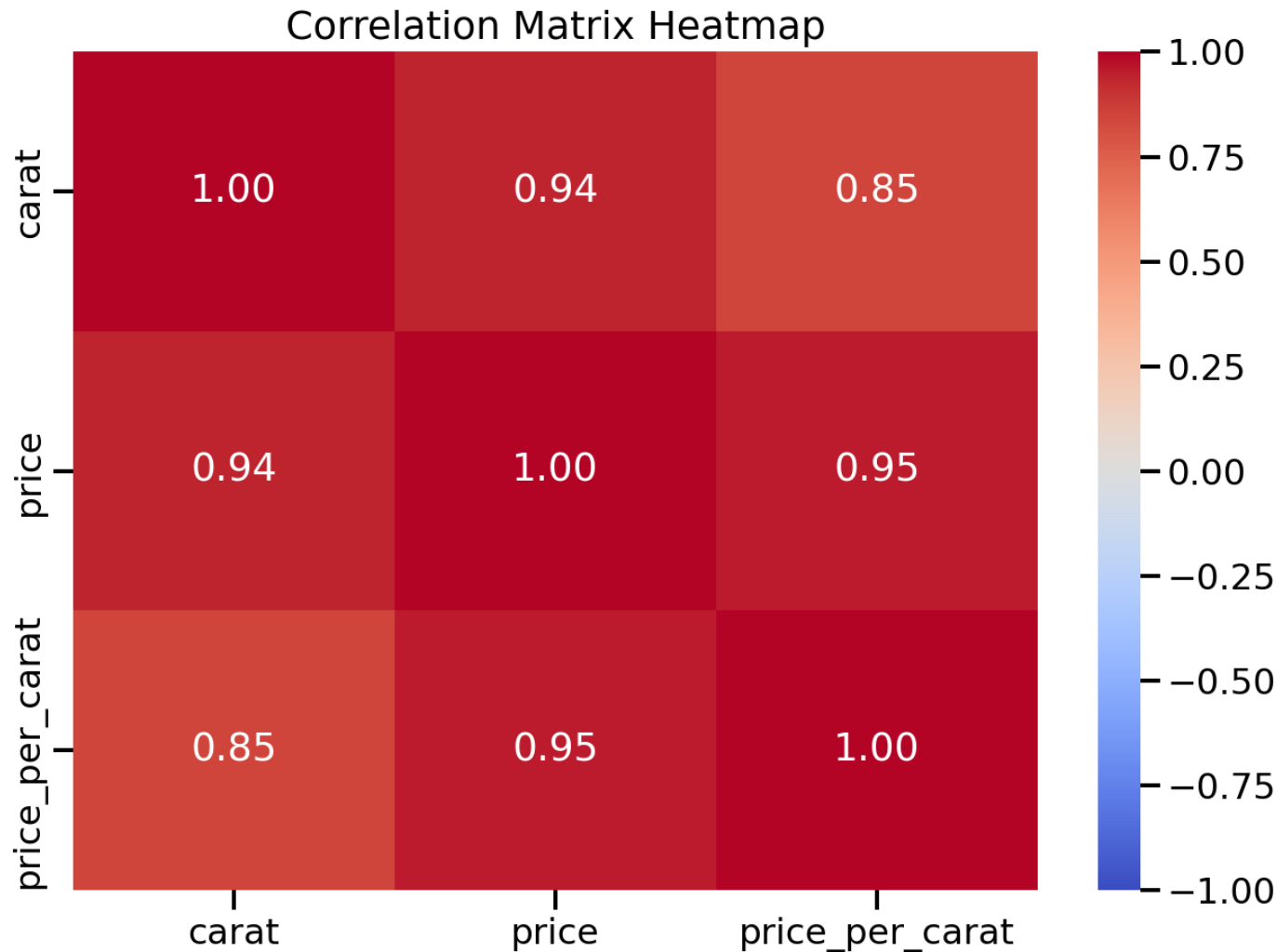
- The price of one carat of each stone, computed row by row
- Median 6,992 SGD per carat
- From 3,190 to 15,850 SGD per carat
- Now there are three numerical columns to compare

Correlation heatmap

```
num_df = df[["carat", "price", "price_per_carat"]]  
corr_matrix = num_df.corr()  
sns.heatmap(corr_matrix, annot=True, fmt=".2f",  
             cmap="coolwarm", vmin=-1, vmax=1)
```

- Every pair of columns gets one correlation
- `vmin=-1, vmax=1` fix the colour scale to the full range
- Red: strong positive, blue: strong negative, white: none

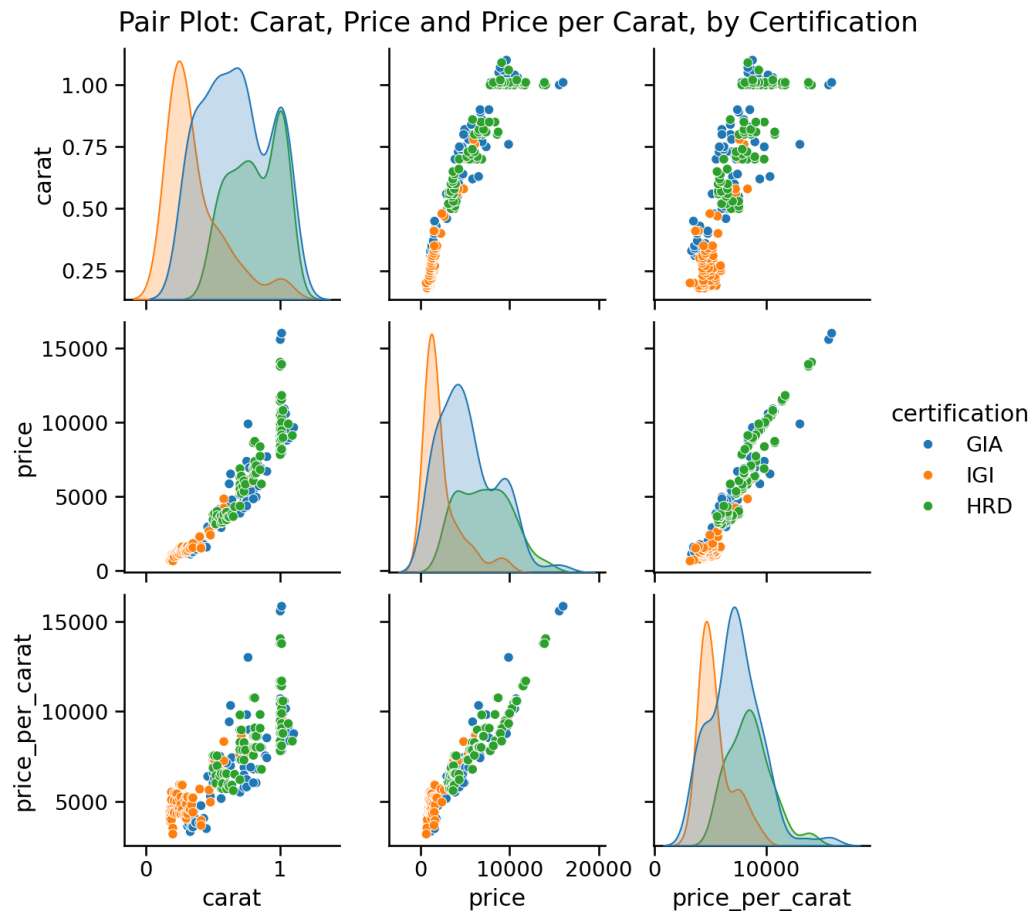
Reading the correlations



- The diagonal is always 1
- carat and price: 0.94
- carat and price per carat: 0.85
- Heavier stones cost more for every carat, not only in total

Pair plot

```
sns.pairplot(df, vars=["carat", "price", "price_per_carat"],  
             hue="certification")
```



- One scatter plot for every pair of columns
- The diagonal: each column alone
- hue= colours the dots by laboratory: a fourth variable
- IGI stones sit in the lower left: small, cheap, cheap per carat

Several charts in one figure

```
fig, axes = plt.subplots(2, 2, figsize=(12, 8))
sns.histplot(df["carat"], bins=20, ax=axes[0, 0])
axes[0, 0].set_title("Histogram of Carat")
plt.tight_layout()
```

- `plt.subplots(2, 2)` creates four empty panels
- `axes[row, column]` picks one, counting from 0; `ax=` draws on it
- In a grid use `axes[...].set_title()`, not `plt.title()`

Summary

- Choose the chart by the types of the variables
- Put ordered categories in order: `.reindex()`, `order=`, `[order]`
- Every chart needs a title and axis labels with units
- Write what you see under every chart
- Charts confirm the numbers of seminar 2 and raise new questions
- A difference between groups may come from another variable